

Consumo de la API Alquiler de Autos en Flutter

Servicios, modelos y conexión con las pantallas ya diseñadas

SENA · Regional Cauca

Centro de Teleinformática y Producción Industrial (CTPI)

Programa: Análisis y Desarrollo de Software (ADSO)

Instructor: Franco Reina

Repositorios y enlaces del proyecto

Recurso	URL
Repositorio Backend	https://github.com/Francorey23/BakendAlquilerAutos.git
Repositorio Frontend	https://github.com/Francorey23/FrontendAlquilerAutosV2_25.git
Backend en Render	https://bakendalquilerautos.onrender.com/
Base de datos Neon	https://neon.tech/home

Antes de escribir código, verifica que tu backend en Render esté activo. Abre la URL en el navegador y confirma que responde API alquiler de autos funcionando correctamente. La primera petición puede tardar hasta un minuto mientras el servidor despierta.

Estructura del proyecto

Los archivos nuevos que crearemos van en estas carpetas dentro de lib/:

```
lib/
|-- controllers/
|   |-- cliente_controller.dart    NUEVO   clase ClienteService
|   |-- autos_controller.dart     NUEVO   clase AutosController
|   |-- alquiler_controller.dart  NUEVO   clase AlquilerService
|-- views/                        YA EXISTEN, se modifican
|   |-- registerScreen.dart
|   |-- loginScreen.dart
|   |-- menuPrincipal.dart
|   |-- detalleVehiculo.dart
|   |-- revisarAlquileres.dart    NUEVO
|-- main.dart
```

A. Registro de usuarios

1. Instalar paquetes

Ejecuta estos dos comandos en la terminal del proyecto:

```
flutter pub add http
flutter pub add shared_preferences
```

http: Biblioteca para realizar solicitudes HTTP (GET, POST, PUT, DELETE).

shared_preferences: Permite guardar el token de sesión en el dispositivo para que persista entre sesiones.

2. Crear el archivo cliente_controller.dart

Crea el archivo en lib/controllers/. Por ahora solo contiene la clase con el método de registro. Los demás métodos los agregaremos después de probar cada uno.

CREAR ARCHIVO

lib/controllers/cliente_controller.dart

```
import 'dart:convert'; // codificar y decodificar JSON
import 'package:http/http.dart' as http; // solicitudes HTTP
import 'package:shared_preferences/shared_preferences.dart';

class ClienteService {
  // IMPORTANTE: cambia esta URL por la de TU backend en Render
  final String baseUrl =
    'https://bakendalquilerautos.onrender.com/api/clientes';

  // POST /api/clientes/registro
  Future<http.Response> registrarCliente(
    String nombre,
    String correo,
    String numLic,
    String password,
  ) async {
    final url = Uri.parse('$baseUrl/registro');
    final response = await http.post(
      url,
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'nombre': nombre,
        'correo': correo,
        'numLic': numLic,
        'password': password,
      })),
    );
    return response;
  }
} // <-- cierre de la clase ClienteService
```

Conceptos clave

Future<http.Response>: El método es asíncrono y devolverá un objeto http.Response cuando termine.

Uri.parse('\$baseUrl/registro'): Construye la URL completa para la solicitud POST.

headers: {'Content-Type': 'application/json'}: Indica al servidor que el contenido viene en formato JSON.

body: jsonEncode({...}): Convierte el mapa de Dart en texto JSON para enviarlo.

Asíncronía (async/await): Las solicitudes a servidores son operaciones lentas y deben manejarse de forma asíncrona para no bloquear la interfaz de usuario.

3. Integrar registrarCliente en registerScreen.dart

Ahora vamos a la vista para poder probar el método. Abre `registerScreen.dart` y agrega:

3a. Import y variables — al inicio de la clase `_RegisterState`

ABRIR Y MODIFICAR**lib/views/registerScreen.dart**

```
// Arriba del archivo, con los otros import
import '../controllers/cliente_controller.dart';

// Dentro de la clase _RegisterState, al inicio
class _registerState extends State<register> {
  final _formKey = GlobalKey<FormState>(); // llave para validar el formulario
  final TextEditingController _nombreController = TextEditingController();
  final TextEditingController _correoController = TextEditingController();
  final TextEditingController _numLicController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  final TextEditingController _passwordControllervalida = TextEditingController();
  final ClienteService clienteService = ClienteService();
```

`TextEditingController`: Controladores que capturan lo que el usuario escribe en cada `TextField`.

`ClienteService`: Instancia del servicio para usar sus métodos.

`GlobalKey<FormState>`: Llave que permite validar todos los campos del formulario con una sola llamada.

3b. Método `registrarCliente` — dentro de la clase, antes del `build`

ABRIR Y MODIFICAR

lib/views/registerScreen.dart

```
// Dentro de _registerState, ANTES del metodo build

void registrarCliente() async {
  // Validar el formulario antes de enviar
  if (!_formKey.currentState!.validate()) return;

  final nombre = _nombreController.text;
  final correo = _correoController.text;
  final numLic = _numLicController.text;
  final password = _passwordController.text;
  final passwordvalida = _passwordControllervalida.text;

  if (password != passwordvalida) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Las contraseñas no coinciden')),
    );
    return;
  }

  try {
    final response = await clienteService.registrarCliente(
      nombre, correo, numLic, password);

    print('Status Code: ${response.statusCode}');
    print('Response Body: ${response.body}');

    if (response.statusCode == 200 || response.statusCode == 201) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Registro exitoso')),
      );
      limpiarCampos();
      irLogin();
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Error al registrar: ${response.body}')),
      );
    }
  } catch (e) {
    print("Error en el registro: $e");
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error en la conexión con el servidor')),
    );
  }
}

void limpiarCampos() {
  setState(() {
    _nombreController.clear();
    _correoController.clear();
    _numLicController.clear();
    _passwordController.clear();
    _passwordControllervalida.clear();
  });
}

void irLogin() {
  Future.delayed(Duration(seconds: 2), () {
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (context) => LoginScreen()),
    );
  });
}
```

3c. Validación en el formulario — dentro del build

Envuelve los campos en un Form con la llave `_formKey`. Agrega validator en al menos un campo como ejemplo. Los demás campos siguen igual.

ABRIR Y MODIFICAR

lib/views/registerScreen.dart

```
// Dentro del build, envolver los campos en Form

Form(
  key: _formKey,
  child: Column(
    children: [
      // Campo con validacion: ejemplo con correo
      TextFormField(
        controller: _correoController,
        keyboardType: TextInputType.emailAddress,
        decoration: InputDecoration(
          labelText: 'Correo electronico',
          prefixIcon: Icon(Icons.email, color: Colors.grey),
          border: OutlineInputBorder(),
        ),
        validator: (valor) {
          if (valor == null || valor.trim().isEmpty) {
            return 'El correo es obligatorio';
          }
          if (!valor.contains('@') || !valor.contains('.')) {
            return 'Escribe un correo valido';
          }
          return null; // null = sin error
        },
      ),
      SizedBox(height: 16),

      // Campo con validacion: contrasena
      TextFormField(
        controller: _passwordController,
        obscureText: true,
        decoration: InputDecoration(
          labelText: 'Contrasena',
          prefixIcon: Icon(Icons.lock, color: Colors.grey),
          border: OutlineInputBorder(),
        ),
        validator: (valor) => (valor == null || valor.length < 6)
          ? 'Minimo 6 caracteres'
          : null,
      ),
      SizedBox(height: 16),

      // Los demas TextFormField con su controller...
      // (nombre, numLic, confirmar contrasena)

      // Boton de registro
      ElevatedButton(
        onPressed: () {
          registrarCliente();
        },
        child: Text('Registrarse'),
      ),
    ],
  ),
)
```

`_formKey.currentState!.validate()`: Ejecuta el validator de **todos** los campos del formulario. Si alguno devuelve un texto, lo muestra como error debajo del campo y la función devuelve `false`. Solo cuando todos devuelven `null`, la función devuelve `true` y el registro continúa.

4. Probar el registro

Ejecuta la app en el emulador o el celular. Llena todos los campos y pulsa **Registrarse**.

Prueba	Esperado
Llenar todos los campos correctamente	201 en consola, SnackBar verde, pasa a login
Dejar el correo vacío	Mensaje rojo debajo del campo: El correo es obligatorio
Contraseña menor a 6 caracteres	Mensaje: Minimo 6 caracteres
Contraseñas que no coinciden	SnackBar: Las contraseñas no coinciden
Registrar el mismo correo otra vez	Error del backend (409 o mensaje de correo duplicado)

Revisa la **Consola de Depuración** en VS Code. El print del Status Code y Response Body te confirman exactamente qué respondió el servidor. Si el código es 404 o ves Cannot POST, la URL está mal.

B. Inicio de sesión

1. Agregar loginCliente en cliente_controller.dart

Abre cliente_controller.dart. Agrega este método **dentro de la clase** ClienteService, después del cierre de registrarCliente y **antes** de la llave } que cierra la clase.

ABRIR Y MODIFICAR

lib/controllers/cliente_controller.dart

```
// Dentro de ClienteService, despues de registrarCliente

// POST /api/clientes/login
Future<Map<String, dynamic>> loginCliente(
  String correo,
  String password,
) async {
  final url = Uri.parse('$baseUrl/login');
  final response = await http.post(
    url,
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({
      'correo': correo,
      'password': password,
    }),
  );

  final responseData = jsonDecode(response.body);

  if (response.statusCode == 200) {
    // Guardar el token en el dispositivo
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('token', responseData['token']);

    return {
      'success': true,
      'mensaje': responseData['mensaje'],
      'token': responseData['token'],
    };
  } else {
    return {
      'success': false,
      'mensaje': responseData['mensaje'] ?? 'Credenciales incorrectas',
    };
  }
}

} // <-- cierre de la clase ClienteService
```

El login del backend devuelve {mensaje, token}. **No** devuelve el objeto cliente. El token queda guardado en el dispositivo con shared_preferences.

2. Integrar loginCliente en loginScreen.dart

ABRIR Y MODIFICAR

lib/views/loginScreen.dart

```
// Arriba, con los otros import
import '../controllers/cliente_controller.dart';
import 'menuPrincipal.dart';

// Dentro de la clase _LoginScreenState, al inicio
final TextEditingController _correoController = TextEditingController();
final TextEditingController _passwordController = TextEditingController();
final ClienteService clienteService = ClienteService();

// Metodo iniciarSesion – dentro de la clase, ANTES del build
void iniciarSesion() async {
  final correo = _correoController.text;
  final password = _passwordController.text;

  final result = await clienteService.loginCliente(correo, password);

  if (result['success']) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Inicio de sesion exitoso')),
    );

    Future.delayed(Duration(seconds: 1), () {
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (context) => MenuPrincipal()),
      );
    });
  } else {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content:
          Text(result['mensaje'] ?? 'Credenciales incorrectas'),
      );
    );
  }
}
```

Agrega los controller en los TextField y conecta el botón:

ABRIR Y MODIFICAR

lib/views/loginScreen.dart

```
// Dentro del build

TextField(
  controller: _correoController,
  decoration: InputDecoration(
    labelText: 'Correo electronico',
    prefixIcon: Icon(Icons.email, color: Colors.grey),
    border: OutlineInputBorder(),
  ),
),
 SizedBox(height: 16.0),
TextField(
  controller: _passwordController,
  obscureText: true,
  decoration: InputDecoration(
    labelText: 'Contraseña',
    prefixIcon: Icon(Icons.lock, color: Colors.grey),
    border: OutlineInputBorder(),
  ),
),
 SizedBox(height: 24.0),
ElevatedButton(
  onPressed: () {
    iniciarSesion();
  },
  child: Text('Iniciar sesion'),
),
```

3. Probar el login

Prueba	Esperado
Login con el correo recién registrado	SnackBar: Inicio de sesion exitoso, navega a MenuPrincipal
Login con contraseña incorrecta	SnackBar: Contraseña incorrecta (401)
Login con correo que no existe	SnackBar: Cliente no encontrado (404)

C. Listar vehículos en MenuPrincipal

1. Crear autos_controller.dart

CREAR ARCHIVO

lib/controllers/autos_controller.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class AutosController {
  // IMPORTANTE: cambia esta URL por la de TU backend
  final String baseUrl =
    'https://bakendaalquilerautos.onrender.com/api/autos';

  // GET /api/autos - trae todos los vehiculos
  Future<List<Map<String, dynamic>>> obtenerAutosDisponibles() async {
    try {
      final url = Uri.parse(baseUrl);
      final response = await http.get(url);

      if (response.statusCode == 200) {
        final List<dynamic> data = jsonDecode(response.body);

        return data.map<Map<String, dynamic>>((auto) {
          final imagen = auto['imagen']?.toString().trim();

          return {
            'id': auto['id'],
            'marca': auto['marca'],
            'modelo': auto['modelo'],
            'anio': auto['anio'],
            'disponibilidad': auto['disponibilidad'],
            'imageUrl': (imagen != null &&
              imagen.isNotEmpty &&
              (imagen.startsWith('http://') ||
                imagen.startsWith('https://')))
              ? imagen
              : 'https://picsum.photos/330/200',
            'precio': auto['valorAlquiler'],
          };
        }).toList();
      } else {
        throw Exception('Error al obtener vehiculos');
      }
    } catch (e) {
      print('Error en obtenerAutosDisponibles: $e');
      return [];
    }
  }
}
```

Explicación del código

`Future<List<Map<String, dynamic>>>`: Retorna una lista de mapas, donde cada mapa representa un auto con sus datos.

`Uri.parse(baseUrl)`: Crea la URL. `await http.get(url)` envía una petición GET y espera la respuesta.

`jsonDecode(response.body)`: Convierte la respuesta JSON en una lista `List<dynamic>`.

`data.map((auto) {...})`: Recorre cada auto, creando un mapa clave-valor con los campos que necesita la vista.

2. Integrar en menuPrincipal.dart

Abre `menuPrincipal.dart`. Agrega el import arriba y las variables dentro de `_MenuPrincipalState`:

ABRIR Y MODIFICAR

lib/views/menuPrincipal.dart

```
// Arriba, con los otros import
import '../controllers/autos_controller.dart';
import 'detalleVehiculo.dart';

// Dentro de _MenuPrincipalState, al inicio
final AutosController autosController = AutosController();
List<Map<String, dynamic>> listaDeAutos = [];
bool isLoading = true;
```

autosController: Instancia que obtiene los datos de los autos.

listaDeAutos: Lista donde se almacenan los autos de la API.

isLoading: Muestra un CircularProgressIndicator mientras carga.

Agrega initState y cargarAutos dentro de la clase, **antes** del build:

ABRIR Y MODIFICAR

lib/views/menuPrincipal.dart

```
// Dentro de _MenuPrincipalState, antes del build

@override
void initState() {
  super.initState();
  cargarAutos();
}

void cargarAutos() async {
  try {
    final autos = await autosController.obtenerAutosDisponibles();
    setState(() {
      listaDeAutos = autos;
      isLoading = false;
    });
  } catch (e) {
    setState(() {
      isLoading = false;
    });
    print('Error al cargar autos: $e');
  }
}
```

Dentro del build, reemplaza la lista quemada por el ListView.builder conectado a la API:

ABRIR Y MODIFICAR

lib/views/menuPrincipal.dart

```
// Dentro del build, donde antes iba la lista estatica

Expanded(
  child: isLoading
    ? Center(child: CircularProgressIndicator())
    : listaDeAutos.isEmpty
      ? Center(child: Text('No hay vehiculos disponibles'))
      : ListView.builder(
        itemCount: listaDeAutos.length,
        itemBuilder: (BuildContext context, int index) {
          final auto = listaDeAutos[index];

          return ListTile(
            leading: ClipRRect(
              borderRadius: BorderRadius.circular(8),
              child: Image.network(
                auto['imageUrl'].toString(),
                width: 50,
                height: 50,
                fit: BoxFit.cover,
                errorBuilder: (context, error, stackTrace) {
                  return Image.network(
                    'https://picsum.photos/330/200',
                    width: 50,
                    height: 50,
                    fit: BoxFit.cover,
                  );
                },
              ),
            ),
            title: Text(
              '${auto['marca']} ${auto['modelo']}',
            ),
            subtitle: Text(
              'Año: ${auto['anio']} - Precio: \${auto['precio']}/dia',
            ),
            trailing: Icon(Icons.arrow_forward_ios),
            onTap: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => DetalleVehiculoScreen(
                    imageUrl: auto['imageUrl'],
                    marca: auto['marca'],
                    modelo: auto['modelo'],
                    anio: auto['anio'],
                    disponibilidad: auto['disponibilidad'],
                    precio: auto['precio'],
                    autoId: auto['id'] ?? 0,
                  ),
                ),
              ).then((_) {
                cargarAutos(); // recarga al volver del detalle
              });
            },
          );
        },
      ),
),
```

Concepto: Navigator.push y el onTap

- `Navigator.push`: Agrega una nueva pantalla a la pila de navegación.
- `context`: Contexto de la aplicación, necesario para navegar.
- `MaterialPageRoute`: Crea una ruta con animación de transición deslizando.
- `builder`: Especifica la pantalla destino (`DetalleVehiculoScreen`).

- Se pasan parámetros obtenidos desde `listaDeAutos[index]`.
- `.then( (_) { cargarAutos(); } )`: Se ejecuta al volver del detalle, recargando la lista con el estado real del backend.

3. Probar

Ejecuta la app, haz login y el `MenuPrincipal` debe mostrar la lista de vehículos del backend. Si está vacía, verifica que haya autos registrados en Neon.

D. Detalle del vehículo

En `detalleVehiculo.dart` definimos los atributos para recibir los datos del auto seleccionado:

ABRIR Y MODIFICAR

`lib/views/detalleVehiculo.dart`

```
class DetalleVehiculoScreen extends StatefulWidget {  
  final String imageUrl;  
  final String marca;  
  final String modelo;  
  final String anio;  
  final int disponibilidad;  
  final int autoId;      // ID del auto para el alquiler  
  final dynamic precio;  
  
  DetalleVehiculoScreen({  
    required this.imageUrl,  
    required this.marca,  
    required this.modelo,  
    required this.anio,  
    required this.disponibilidad,  
    required this.autoId,  
    this.precio,  
  });  
};
```

El constructor recibe los datos cuando se navega desde el `onTap` del `MenuPrincipal`.

E. Servicio de Alquiler

El backend usa JWT. El clienteId **no se envía** en el body: el backend lo obtiene del token. Las rutas de alquiler exigen el header Authorization: Bearer <token>.

1. Crear alquiler_controller.dart

CREAR ARCHIVO

lib/controllers/alquiler_controller.dart

```

import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';

class AlquilerService {
  final String baseUrl =
    'https://bakendaalquilerautos.onrender.com/api/alquiler';

  // Arma la cabecera con el token para rutas protegidas
  Future<Map<String, String>> _headersConToken() async {
    final prefs = await SharedPreferences.getInstance();
    final token = prefs.getString('token') ?? '';
    return {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer $token',
    };
  }

  // POST /api/alquiler – registrar un alquiler
  Future<Map<String, dynamic>> registrarAlquiler(
    int autoId, String fechaInicio, String fechaFin) async {
    try {
      final url = Uri.parse(baseUrl);
      final headers = await _headersConToken();
      final response = await http.post(
        url,
        headers: headers,
        body: jsonEncode({
          'autoId': autoId,
          'fechaInicio': fechaInicio,
          'fechaFin': fechaFin,
        })),
      ).timeout(const Duration(seconds: 15));

      print('=== RESPUESTA SERVIDOR ===');
      print('Status: ${response.statusCode}');
      print('Body: ${response.body}');

      final data = jsonDecode(response.body);

      if (response.statusCode == 200 || response.statusCode == 201) {
        if (data is Map && data.containsKey('mensaje')) {
          return {'success': false, 'message': data['mensaje']};
        }
        return {'success': true, 'data': data};
      } else {
        return {
          'success': false,
          'message': data['mensaje'] ?? 'Error al registrar el alquiler',
        };
      }
    } catch (e) {
      print('Error en registrarAlquiler: $e');
      return {'success': false, 'message': 'Error de conexion con el servidor'};
    }
  }

  // GET /api/alquiler/historial – solo los del cliente autenticado
  Future<List<dynamic>> obtenerHistorial() async {
    final headers = await _headersConToken();
    final url = Uri.parse('$baseUrl/historial');
    final response = await http.get(url, headers: headers);

    if (response.statusCode == 200) {
      return jsonDecode(response.body);
    } else {

```

```

        throw Exception('Error al cargar alquileres');
    }
}

// PUT /api/alquiler/devolver/:id - devolver vehiculo
Future<Map<String, dynamic>> devolverVehiculo(int idAlquiler) async {
    final headers = await _headersConToken();
    final url = Uri.parse('$baseUrl/devolver/$idAlquiler');
    final response = await http.put(url, headers: headers);

    final data = jsonDecode(response.body);
    if (response.statusCode == 200) {
        return {'success': true, 'mensaje': data['mensaje']};
    } else {
        return {
            'success': false,
            'mensaje': data['mensaje'] ?? 'Error al devolver vehiculo',
        };
    }
}

} // <-- cierre de la clase AlquilerService

```

2. Integrar alquiler en detalleVehiculo.dart

Agrega el import y las variables de estado dentro de la clase:

ABRIR Y MODIFICAR

lib/views/detalleVehiculo.dart

```

// Arriba, con los otros import
import '../controllers/alquiler_controller.dart';

// Dentro de _DetalleVehiculoScreenState, al inicio
bool disponible = true;
bool cargando = false;
final AlquilerService alquilerService = AlquilerService();

@override
void initState() {
    super.initState();
    // Inicializar disponibilidad basada en el dato del vehiculo
    disponible = widget.disponibilidad == 1;
}

```

Agrega las funciones de alquiler **dentro de la clase**, después de initState y **antes** del build:

ABRIR Y MODIFICAR

lib/views/detalleVehiculo.dart

```
// Dentro de _DetalleVehiculoScreenState, antes del build

void alquilarVehiculo() async {
  final String fechaInicio = DateTime.now().toUtc().toIso8601String();
  final String fechaFin = DateTime.now().toUtc()
    .add(Duration(days: 7))
    .toIso8601String();

  print('Auto ID: ${widget.autoId}');
  print('Fecha inicio: $fechaInicio');
  print('Fecha fin: $fechaFin');

  try {
    final response = await alquilerService.registrarAlquiler(
      widget.autoId, fechaInicio, fechaFin);

    if (!mounted) return;
    print('Respuesta: $response');

    if (response['success']) {
      setState(() {
        disponible = false;
      });
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Alquiler registrado con exito')),
      );
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Error: ${response['message']}')),
      );
    }
  } catch (e) {
    print('Error al registrar alquiler: $e');
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error de conexion: $e')),
    );
  }
}

void confirmarAlquiler() {
  showDialog(
    context: context,
    builder: (BuildContext context) {
      return AlertDialog(
        title: Text('Confirmar Alquiler'),
        content: Text('Esta seguro de que desea alquilar este vehiculo?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.of(context).pop(),
            child: Text('Cancelar'),
          ),
          TextButton(
            onPressed: () {
              Navigator.of(context).pop();
              alquilarVehiculo();
            },
            child: Text('Aceptar'),
          ),
        ],
      );
    },
  );
}
```

Reemplaza el botón de alquiler en el build:

ABRIR Y MODIFICAR

lib/views/detalleVehiculo.dart

```
// Dentro del build, el boton de alquiler

ElevatedButton.icon(
  style: ElevatedButton.styleFrom(
    backgroundColor: disponible ? boton : Colors.grey,
    foregroundColor: Colors.white,
    minimumSize: const Size(double.infinity, 50),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
  onPressed: (!disponible || cargando)
    ? null
    : () => confirmarAlquiler(),
  icon: cargando
    ? const SizedBox(
        width: 18, height: 18,
        child: CircularProgressIndicator(
          color: Colors.white, strokeWidth: 2),
      )
    : const Icon(Icons.directions_car),
  label: Text(
    disponible ? 'Solicitar alquiler' : 'No disponible',
    style: const TextStyle(fontSize: 16),
  ),
),
```

Etiqueta visual de disponibilidad (reemplaza el Chip anterior):

ABRIR Y MODIFICAR

lib/views/detalleVehiculo.dart

```
// Donde antes estaba el Chip de disponibilidad

Container(
  padding: EdgeInsets.symmetric(horizontal: 12, vertical: 6),
  decoration: BoxDecoration(
    color: disponible ? Colors.green : Colors.red,
    borderRadius: BorderRadius.circular(20),
  ),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Icon(disponible ? Icons.check : Icons.close,
        color: Colors.white),
      SizedBox(width: 5),
      Text(
        disponible ? 'Disponible' : 'No disponible',
        style: TextStyle(color: Colors.white),
      ),
    ],
  ),
),
```

3. Probar el alquiler

Prueba	Esperado
Pulsar Solicitar alquiler en un auto disponible	Diálogo de confirmación, luego SnackBar de éxito, botón cambia a No disponible
Intentar alquilar el mismo auto otra vez	Error: Este auto no esta disponible (400)
Volver al menú principal	El auto aparece con la disponibilidad actualizada

F. Historial de alquileres

El servicio obtenerHistorial() ya está en alquiler_controller.dart. Ahora creamos la vista.

La ruta es GET /api/alquiler/historial (protegida con token). El backend filtra por el cliente autenticado. En el historial, el auto anidado viene bajo la clave Auto con mayúscula (así lo devuelve Sequelize con el include).

1. Crear revisarAlquileres.dart

CREAR ARCHIVO

lib/views/revisarAlquileres.dart

```

import 'package:flutter/material.dart';
import '../controllers/alquiler_controller.dart';

class RevisarAlquileresScreen extends StatefulWidget {
  const RevisarAlquileresScreen({super.key});

  @override
  _RevisarAlquileresScreenState createState() =>
    _RevisarAlquileresScreenState();
}

class _RevisarAlquileresScreenState
  extends State<RevisarAlquileresScreen> {
  List alquileres = [];
  bool isLoading = true;

  @override
  void initState() {
    super.initState();
    cargarAlquileres();
  }

  void cargarAlquileres() async {
    try {
      final data = await AlquilerService().obtenerHistorial();
      setState(() {
        alquileres = data;
        isLoading = false;
      });
    } catch (e) {
      print(e);
      setState(() {
        isLoading = false;
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Mis Alquileres'),
      ),
      body: isLoading
        ? const Center(child: CircularProgressIndicator())
        : alquileres.isEmpty
          ? const Center(
              child: Text('No tienes alquileres registrados'))
          : ListView.builder(
              itemCount: alquileres.length,
              itemBuilder: (context, index) {
                final item = alquileres[index];
                final auto = item['Auto']; // con mayuscula

                return Card(
                  margin: const EdgeInsets.all(10),
                  child: ListTile(
                    leading: Image.network(
                      auto?['imagen'] ??
                        'https://picsum.photos/200',
                      width: 60,
                      fit: BoxFit.cover,
                      errorBuilder: (context, error, stackTrace) {
                        return Image.network(
                          'https://picsum.photos/200',
                          width: 60,

```

```

        fit: BoxFit.cover,
      );
    },
  ),
  title: Text(
    '${auto?['marca'] ?? ''} '
    '${auto?['modelo'] ?? ''}',
    style: TextStyle(
      fontWeight: FontWeight.bold),
  ),
  subtitle: Column(
    crossAxisAlignment:
      CrossAxisAlignment.start,
    children: [
      Text('Ano: ${auto?['anio'] ?? ''}'),
      Text(
        'Inicio: ${item['fechaInicio']}',
      ),
      Text('Fin: ${item['fechaFin']}',
      ),
      Text('Estado: ${item['estado']}',
      ),
    ],
  ),
),
),
);
},
),
);
}
}

```

2. Conectar desde el Drawer

ABRIR Y MODIFICAR

lib/views/menuDrawerPerfil.dart

```

// Dentro del ListView del Drawer

ListTile(
  leading: Icon(Icons.history),
  title: Text('Revisar alquileres'),
  onTap: () {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => const RevisarAlquileresScreen(),
      ),
    );
  },
),
),

```

3. Probar

Prueba	Esperado
Abrir Mis Alquileres después de haber alquilado	Lista con el auto anidado (marca, modelo, fechas, estado)
Sin alquileres	Mensaje: No tienes alquileres registrados

G. Generar el APK

Configurar la versión en pubspec.yaml:

```
version: 1.0.0+1
```

Generar:

```
flutter build apk --release
```

El APK queda en:

```
build/app/outputs/flutter-apk/app-release.apk
```

Verificación final

- ☐ La URL del backend aparece una sola vez por controller
- ☐ El registro crea el cliente y la consola muestra 201
- ☐ El login guarda el token y navega al menú
- ☐ El menú carga los autos desde la API
- ☐ Al alquilar, el botón cambia a No disponible
- ☐ Al volver al menú, la lista se refresca
- ☐ El historial muestra los alquileres con el auto anidado
- ☐ No quedan print de depuración con datos del usuario